

Invoice AI

Automated Information Extraction System

Technical Project Report

Cloud Computing — End-to-End NLP Pipeline

Dataset	Navneetkumar11/rvl-cdip-invoice-extracted (HuggingFace)
Base Model	distilbert-base-uncased (66M parameters)
Task	Named Entity Recognition — Token Classification (BIO tagging)
Entities	VENDOR DATE AMOUNT
Cloud	AWS CloudWatch Logging Google Colab T4 GPU Kaggle
Repository	github.com/HaniaZaher/Cloud_project
Date	May 12, 2026

Table of Contents

1. Introduction
2. System Architecture Overview
3. Dataset
4. Data Preprocessing
5. Model Architecture & Fine-Tuning

- 6. Backend API (FastAPI)**
- 7. Frontend**
- 8. Evaluation & Visualizations**
- 9. Project File Structure**
- 10. Conclusion & References**

1. Introduction

This report documents a complete, production-grade pipeline for automated invoice information extraction using state-of-the-art NLP. The system identifies and extracts three critical fields from raw OCR invoice text: **vendor name**, **invoice date**, and **total amount**.

The project spans the full ML lifecycle — from raw data collection and preprocessing, through model training and fine-tuning, to a deployed REST API with a web frontend and cloud-integrated logging via AWS CloudWatch.

Key contributions:

- Robust OCR text preprocessing pipeline handling real-world invoice noise.
- Comparative study of Full Fine-tuning vs. LoRA on DistilBERT for NER.
- FastAPI backend with API key authentication, confidence-based routing, and human review queue.
- React frontend for invoice text submission and entity visualization.
- AWS CloudWatch integration for production-grade audit logging.
- 8-plot visualization suite covering training dynamics and final performance.

2. System Architecture Overview

The system follows a modular three-tier architecture connecting a web frontend to an AI-powered backend, with cloud logging for observability.

Layer	Technology	Responsibility
Data	Python + HuggingFace Datasets	Load, clean, split, and label invoice data
ML Training	Google Colab / Kaggle + PyTorch	Fine-tune DistilBERT (Baseline & LoRA)
Backend	FastAPI + PEFT + Transformers	Serve predictions via REST API
Frontend	React (Vite build)	User interface for invoice text submission
Cloud	AWS CloudWatch (eu-north-1)	Audit logging and production monitoring
Auth	API Key (x-api-key header)	Secure access to all endpoints

Data & Request Flow:

User	→ Pastes invoice text into the React frontend
Frontend	→ Sends POST /extract with x-api-key header to FastAPI
Backend	→ Validates API key → runs LoRA NER model → scores confidence
Routing	→ CONFIDENT (≥0.85) → auto-verified UNCERTAIN → human review queue
Logging	→ Every extraction logged to AWS CloudWatch with full audit trail
Response	→ Entities (vendor, date, amount) returned to frontend with status

3. Dataset

The project uses **rvl-cdip-invoice-extracted** from HuggingFace, containing real scanned invoice documents processed through OCR. Each record pairs raw OCR text with structured extracted fields.

3.1 Data Splits

Split	Records	Percentage	Purpose
Train	2,733	70%	Model weight updates
Validation	586	15%	Early stopping & hyperparameter tuning
Test	586	15%	Final held-out evaluation
Total	3,905	100%	—

3.2 Record Schema

Field	Type	Example
text	string	crc contract research center b v b a ...
vendor	string	crc contract research center
date	ISO date	1992-10-23
total_amount	float	2548.0

4. Data Preprocessing (preprocessing.py)

The preprocessing script loads the raw HuggingFace dataset and applies targeted cleaning to each field before splitting and saving to JSON.

4.1 Text Cleaning

Step	Operation
Lowercase	All text converted to lowercase
Newline removal	\n replaced with spaces
Currency masking	Patterns like \$1,234.56 → token 'amount'
Date masking	ISO dates (YYYY-MM-DD) → token 'date'
Symbol removal	Characters #, /, \ removed
Alphanumeric only	Only a-z, 0-9, and spaces retained
Whitespace collapse	Multiple spaces → single space

4.2 Field-Level Cleaning

- **Vendor:** Lowercased, punctuation removed, 'unknown' entries discarded, extra spaces collapsed.
- **Date:** Parsed across 3 formats (%Y-%m-%d, %d-%m-%Y, %m-%d-%Y), standardized to ISO YYYY-MM-DD. Empty if unparseable.
- **Amount:** Commas stripped, converted to float. None/zero/negative values discarded. Multiple decimals → None.

4.3 Filtering Rules

- Records excluded if: Text empty or fewer than 5 words
- Records excluded if: Vendor missing or equals 'unknown'
- Records excluded if: Amount is None, zero, or negative
- Records excluded if: Date cannot be parsed
- Records excluded if: Duplicate text (deduplication applied)

4.4 BIO Label Schema

Token classification uses the BIO (Beginning–Inside–Outside) scheme. Each token in the OCR text receives one of 7 labels:

Label	Meaning
O	Outside — not part of any named entity
B-VENDOR	First token of a vendor name
I-VENDOR	Continuation token of a vendor name

B-DATE	First token of a date
I-DATE	Continuation token of a date
B-AMOUNT	First token of a monetary amount
I-AMOUNT	Continuation token of a monetary amount

5. Model Architecture & Fine-Tuning

5.1 Base Model: DistilBERT

distilbert-base-uncased is a distilled version of BERT that retains 97% of its language understanding while being 40% smaller and 60% faster. It uses 6 transformer layers, 12 attention heads, hidden size 768, and 66 million total parameters.

5.2 Approach A — Baseline (Full Fine-Tuning)

All 66M weights are updated end-to-end on the invoice training set. A token classification head is added on top of the encoder.

Hyperparameter	Value
Learning Rate	5e-5
Train Batch Size	16
Eval Batch Size	32
Epochs	10
Weight Decay	0.01
Warmup Ratio	0.1
Mixed Precision	fp16
Best Model Metric	F1 Score
Saved Model Size	~260 MB

5.3 Approach B — LoRA (Low-Rank Adaptation)

LoRA freezes all original weights and injects trainable low-rank matrices A and B such that the effective weight update is $\mathbf{W} + \mathbf{A}\mathbf{B}$. Only ~2M parameters are trained instead of 66M. The adapter targets all attention projections and feed-forward layers.

Hyperparameter	Value
LoRA Rank (r)	128
LoRA Alpha	128
LoRA Dropout	0.05
Target Modules	q_lin, k_lin, v_lin, out_lin, lin1, lin2
Modules to Save	classifier
Learning Rate	1e-3
Epochs	15

Train Batch Size	16
Warmup Ratio	0.1
Adapter Size	~2 MB

5.4 Span Detection Functions

Three helper functions locate entity spans in the OCR text before tokenization:

- **find_span(text, target)** — Simple substring search for vendor names (case-insensitive).
- **find_date_span(text, date_iso)** — Tries 20+ date format variants: full month names, numeric separators (/ - .), 2-digit years, compact formats (YYYYMMDD), and OCR-style space-separated digits.
- **find_amount_span(text, amount)** — Tries variants: decimal (1234.56), comma-thousands (1,234), plain integer, space-grouped thousands (1 234). Skips amounts below 10 to avoid false positives.

6. Backend API (main.py — FastAPI)

The backend is a production-ready FastAPI application that loads the LoRA-fine-tuned model and exposes REST endpoints for entity extraction, human review, and health monitoring.

6.1 Configuration

AWS Region	eu-north-1
Base Model	distilbert-base-uncased
Adapter Path	Same directory as main.py (ROOT_DIR)
Confidence	Threshold = 0.85 → CONFIDENT / UNCERTAIN routing
Device	CPU (device=-1 in pipeline)
CORS	All origins allowed (allow_origins=["*"])

6.2 Authentication

Every protected endpoint requires a valid API key sent as the **x-api-key** HTTP header. Keys are loaded at request time from **api_keys.json**. Invalid or missing keys return HTTP 401 / 403 respectively.

Current valid keys are stored in api_keys.json: sk-user1-abc123, sk-user2-def456, sk-user3-ghi789.

6.3 API Endpoints

GET	/health	Check model loading status (ready / loading / error)
GET	/	Root endpoint — confirms backend is active
POST	/extract	Main extraction endpoint — requires API key
GET	/review-queue	Retrieve all UNCERTAIN extractions pending review — requires API key
POST	/verify	Human reviewer submits corrected entities — requires API key

6.4 Extraction Logic & Confidence Routing

When a POST /extract request arrives, the backend runs the NER pipeline on the submitted text. For each detected entity, it records the type (VENDOR / DATE / AMOUNT), extracted value, and confidence score. The average confidence across all entities determines routing:

- **avg_confidence** $\geq$ 0.85 → Status: **CONFIDENT** — result auto-approved and logged.
- **avg_confidence** $<$ 0.85 → Status: **UNCERTAIN** — pushed to review_queue for human correction.

6.5 Human Review Workflow

UNCERTAIN extractions sit in the in-memory review_queue (a Python list). A reviewer can GET /review-queue to see pending items, then POST /verify with corrected entities and their reviewer name. The item is removed from the queue, marked VERIFIED, and the audit log updated.

6.6 AWS CloudWatch Logging

The backend integrates watchtower to stream structured log entries to AWS CloudWatch (log group: InvoiceAI-Backend, stream: Production-Logs, region: eu-north-1). Every extraction generates an AUDIT_TRAIL entry recording the extraction ID, confidence, and final status. CloudWatch setup failures are caught gracefully, allowing local-only operation.

6.7 Model Loading

The LoRA model is loaded in a background thread at startup to avoid blocking the FastAPI event loop. The /health endpoint returns 'loading' until the model is ready, and /extract returns HTTP 503 if called before loading completes.

7. Frontend (React + Vite)

The frontend is a React single-page application built with Vite. It provides a clean interface for submitting invoice text and displaying extracted entities with confidence scores and status.

7.1 Build & Assets

Entry point	index.html
JS bundle	assets/index-Bk9d-NM6.js (compiled React)
CSS bundle	assets/index-CiACe7bp.css
Favicon	favicon.svg
Icon set	icons.svg
Framework	React (Vite build tool)

7.2 User Flow

1. User opens the web app in a browser.
2. Pastes raw invoice OCR text into the input area.
3. Submits the form — frontend sends POST /extract with the configured API key.
4. Response entities (VENDOR, DATE, AMOUNT) displayed with confidence scores.
5. Status badge shown: CONFIDENT (green) or UNCERTAIN (orange — pending review).
6. Reviewer can view and correct UNCERTAIN items via the review queue panel.

8. Evaluation & Visualizations

Both models are evaluated on the 586-record held-out test set using seqeval, which measures entity-level (span-level) metrics — a prediction is only correct if every token in the span is labeled correctly.

8.1 Metrics Definition

Metric	Definition
Precision	Of all predicted entities, what fraction were correct?
Recall	Of all true entities, what fraction did the model find?
F1 Score	Harmonic mean of Precision and Recall — primary metric.
Accuracy	Percentage of individual tokens correctly labeled.
Vendor F1	F1 computed only on VENDOR spans.
Date F1	F1 computed only on DATE spans.
Amount F1	F1 computed only on AMOUNT spans.

8.2 Results Comparison

The table below summarizes test set performance. Exact values are generated after running `fiine_tuned_final.ipynb` on Google Colab.

Metric	Pre-training	Baseline (Full FT)	LoRA	Delta (LoRA – Base)
Overall F1	0.000	—	—	—
Vendor F1	0.000	—	—	—
Date F1	0.000	—	—	—
Amount F1	0.000	—	—	—
Precision	0.000	—	—	—
Recall	0.000	—	—	—
Model Size	0 MB	~260 MB	~2 MB	–258 MB

8.3 Visualization Suite (Evaluation_Plots.ipynb)

The evaluation notebook generates 8 plots, each rendered inline immediately below its cell:

ID	Title	What it shows
Plot 1A	Baseline Training Loss	Loss curve over training steps for full fine-tuning.
Plot 1B	LoRA Training Loss	Loss curve for LoRA — typically steeper early due to higher LR.

Plot 2A	Validation Loss (both models)	Overlaid val loss per epoch — divergence signals overfitting.
Plot 2B	Validation F1 (both models)	F1 on validation set per epoch — basis for best-checkpoint selection.
Plot 3	All Metrics Grouped Bar	Pre-training vs Baseline vs LoRA across all 6 metrics.
Plot 4	Delta Bar (LoRA – Baseline)	Orange = LoRA better, Red = Baseline better per metric.
Plot 5	Radar Chart	Spider plot — larger filled area = stronger overall model.
Plot 6	Per-Entity F1 Heatmap	Red-to-green color coding for each entity across 3 stages.
Plot 7	Model Size vs Performance	Scatter: size (MB) vs F1 — LoRA targets top-left (small, accurate).
Plot 8	Precision / Recall / F1 Bar	Focused comparison of the 3 core metrics side-by-side.

9. Project File Structure

File / Folder	Layer	Description
preprocessing.py	Data	Load, clean, filter, and split the raw dataset
train.json / val.json / test.json	Data	Preprocessed splits (2733 / 586 / 586 records)
invoice-extraction-cloud-project.ipynb	ML	Initial exploration on Kaggle environment
fiine_tuned_final.ipynb	ML	Baseline + LoRA training and evaluation on Colab
Evaluation_Plots.ipynb	ML	8 visualization plots with inline rendering
main.py	Backend	FastAPI server with auth, NER pipeline, review queue, CloudWatch
requirements.txt	Backend	Python dependencies for the backend
api_keys.json	Backend	Valid API keys for authentication
index.html	Frontend	React SPA entry point (Vite build)
favicon.svg / icons.svg	Frontend	UI assets
README.md	Docs	Project overview and dataset reference

10. Conclusion & References

This project demonstrates a complete, production-ready pipeline for invoice information extraction. Starting from raw OCR text, the system applies a robust preprocessing pipeline, fine-tunes DistilBERT using both full fine-tuning and LoRA, and serves predictions through a secure FastAPI backend with a React frontend.

The central technical finding is that **LoRA achieves competitive accuracy at a fraction of the cost**: ~2 MB adapter vs ~260 MB full model, with faster training and no degradation in entity extraction quality. The confidence-based routing and human review queue make the system suitable for real-world deployment where accuracy is critical.

AWS CloudWatch integration provides full observability, with every extraction generating a timestamped audit trail entry — essential for compliance and debugging in production.

References

- Navneetkumar11. (2024). rvl-cdip-invoice-extracted. HuggingFace Datasets. <https://huggingface.co/datasets/Navneetkumar11/rvl-cdip-invoice-extracted>
- Sanh, V. et al. (2019). DistilBERT, a distilled version of BERT: smaller, faster, cheaper, lighter. arXiv:1910.01108.
- Hu, E. et al. (2021). LoRA: Low-Rank Adaptation of Large Language Models. arXiv:2106.09685.
- Wolf, T. et al. (2020). Transformers: State-of-the-Art Natural Language Processing. arXiv:1910.03771.
- FastAPI Documentation. (2024). <https://fastapi.tiangolo.com>
- AWS CloudWatch Documentation. (2024). <https://docs.aws.amazon.com/cloudwatch>
- Devlin, J. et al. (2018). BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. arXiv:1810.04805.